Single large prime on the genus-5 cover: a confound, a baseline fix, and an exploratory crossover

Study `research/humbert_genus5_cover_20261021/lp_20261027` · 2026-10-27 · follows PR #2580

Scope. Toy fields only (q prime, $2003 \le q \le 12007$), the iota-folded index calculus on the descended genus-5 curve C of PR #2580. Two preregistered campaigns, `PROTOCOL.md` (v1, with amendments 1 and 2) and `PROTOCOL_v2.md`, each frozen before its data. Same instance per (q, seed) in every arm; arm order rotated; an A/A arm in each campaign. Rho is measured once per instance; every arm is compared with that instance's expected-steps rho $\sqrt{\pi r/2}$ × measured s/step.

Host. 4-vCPU Intel Xeon @ 2.10 GHz shared cloud VM, no isolation (L0). This deviates from the default isolab-eks route (AGENTS.md §9a), which was not reachable from the session. Nothing else ran on the host during the campaigns, except one overlap whose rows were set aside (amendment 2).

Arms. `iota`: the merged baseline. `lp:`α: small factor base of a seeded fraction α of the x-values; a relation whose residual has exactly one point outside it is a partial, and two partials sharing that large prime combine into a full relation; residuals with two or more such points are dropped; large-prime logs come from their partials after the linear algebra, and the descent may use any point with a known log. `precheck`: never start a Wiedemann solve while a factor-base column appears in no relation.

Correctness. Every one of the 200 runs (120 in v1, 80 in v2) solved the DLP and passed the pushforward ($8P$), $T(P) \neq 0$, iota-pair and $X_\infty = 4c$ checks. The refactored `iota` arm reproduces a recorded PR #2580 run exactly (q = 1009, seed 1).

1. Campaign v1: large primes against the merged baseline

q	A/A median [min, max]	lp:0.5 median [min, max]	lp:0.5 faster	lp:0.35 median [min, max]	lp:0.35 faster
2003	1.015 [0.978, 1.056]	0.61 [0.50, 1.04]	1/5	0.28 [0.26, 0.51]	0/5
3001	0.979 [0.914, 1.010]	0.62 [0.61, 1.65]	1/5	0.32 [0.29, 0.95]	0/5
4001	0.990 [0.970, 1.088]	0.71 [0.66, 3.67]	2/5	0.37 [0.35, 2.35]	1/5
6007	0.990 [0.975, 1.007]	0.82 [0.77, 3.40]	1/5	0.44 [0.44, 1.97]	1/5
8009	1.006 [0.989, 1.046]	2.10 [0.94, 3.67]	3/5	1.23 [0.51, 2.13]	3/5
12007	0.994 [0.991, 1.043]	1.11 [0.67, 6.99]	4/5	0.69 [0.64, 3.85]	2/5

Table 1: Paired ratio $t_{\text{ic(iota)}}/t_{\text{ic(arm)}}$ per instance, 5 seeds per q (`results/campaign.jsonl`); above 1 the arm is faster.

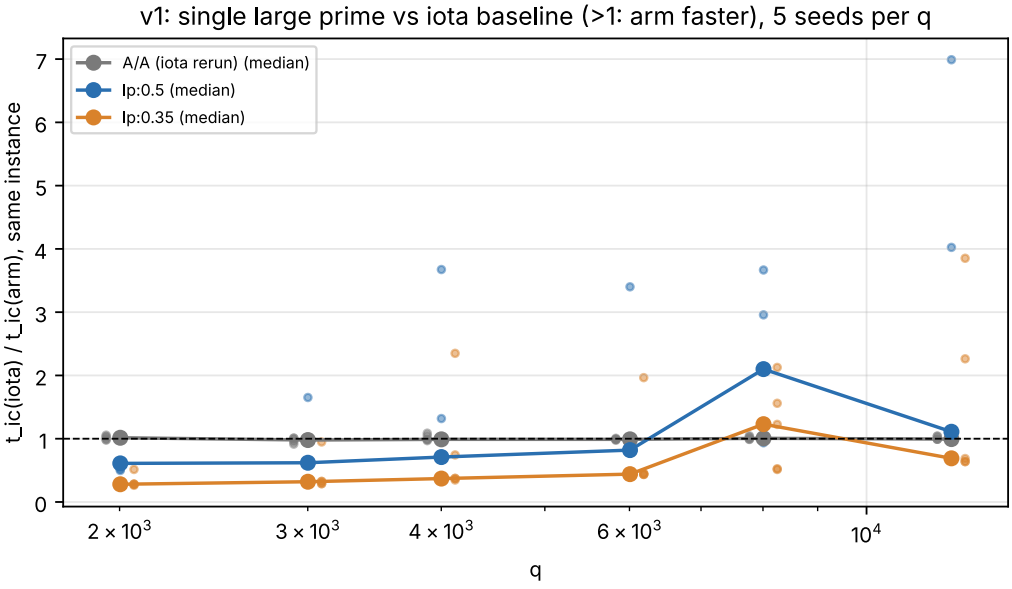


Figure 1: v1 ratios against the merged baseline; points are instances.

Predictions. P1 (lp:0.5 faster on at least 4/5 seeds at $q = 8009$): **falsified**, 3/5. P2 (median ratio at most 1 at $q = 2003$): holds, 0.61. P3 (ratio rises with q): holds in the medians.

Confound (amendment 2). Every lp:0.5 win at $q \ge 4001$ is on an instance where the baseline needed 3–10 linear-algebra rounds: each round collected about 5 % more relations and reran the whole Wiedemann solve, because some factor-base column was still in no relation. On every instance where the baseline needed one round, lp:0.5 was slower (0.66–0.95). The A/A arm

repeats the same rounds exactly, so this is a deterministic property of the baseline's retry policy, not noise and not a property of large primes.

2. Campaign v2: the baseline fix, and large primes against it

q	M1 precheck vs v1, median [min, max]	multi-round: precheck faster	A/A median [min, max]	M2 lp:0.5 vs fixed baseline, median [min, max]	lp:0.5 faster	IC / rho-exp: v1, precheck, lp:0.5	n
4001	1.06 [0.92, 5.06]	2/2	0.968 [0.941, 1.023]	0.71 [0.65, 0.77]	0/5	5.84, 1.52, 2.02	5
6007	1.04 [0.98, 3.95]	1/1	0.964 [0.955, 0.992]	0.78 [0.76, 0.93]	0/5	2.15, 2.06, 2.64	5
8009	2.17 [1.00, 3.46]	3/3	1.014 [0.992, 1.027]	0.97 [0.89, 1.04]	2/5	3.43, 2.45, 2.69	5
12007	0.99 [0.97, 4.87]	2/2	0.994 [0.968, 1.004]	1.08 [1.06, 1.33]	5/5	4.03, 3.32, 3.08	5

Table 2: Paired ratios per instance (`results/v2.jsonl`, per-instance table in `results/summary_v2.md`).

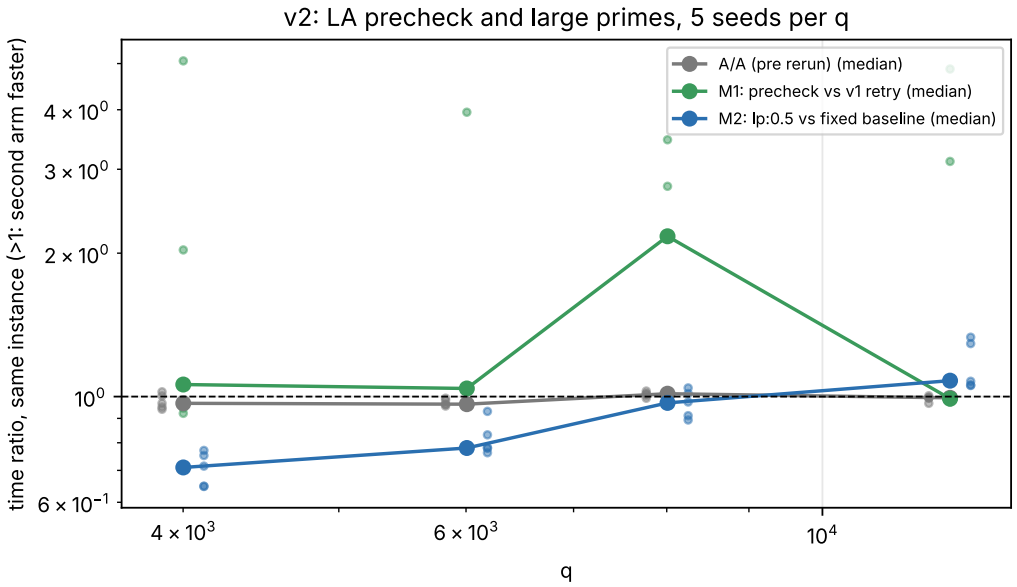


Figure 2: v2: precheck against the v1 retry policy (M1), large primes against the fixed baseline (M2), and the A/A floor.

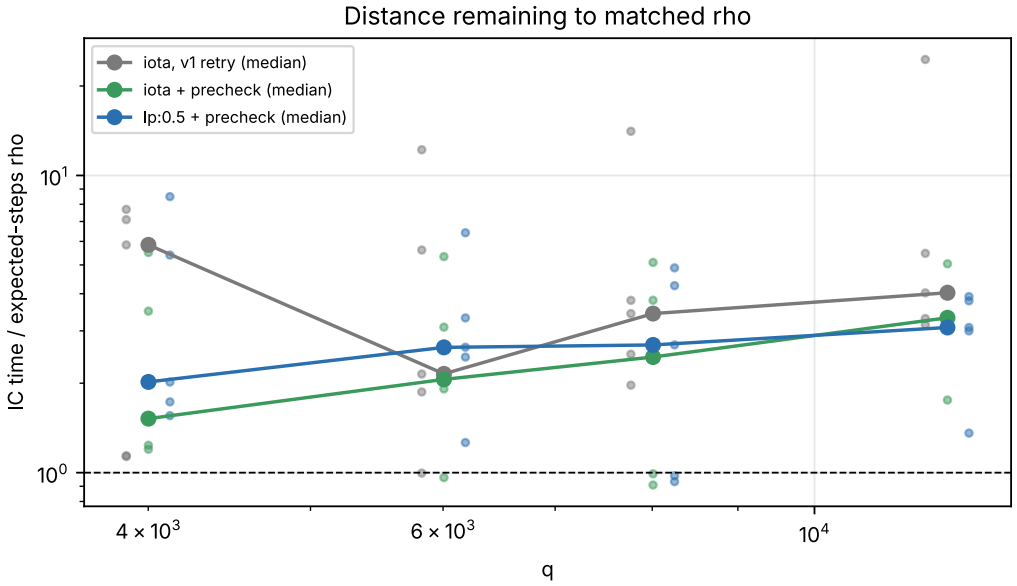


Figure 3: Distance remaining to matched rho, medians of 5 instances.

Predictions. Q1, precheck faster on every multi-round instance: holds, 8/8 (2.0–5.1×; e.g. $q = 12007$ seed 5, 224.5 s to 46.1 s, 7 rounds to 1). Q1, one-round instances within the A/A band: **fails on one instance** ($q = 4001$ seed 2, 0.92 against an A/A floor of 0.941 at that q). On one-round instances the only difference is a column scan of the relation rows, so this is most likely noise, but Q1 is falsified as written. Q2, lp:0.5 not faster against the fixed baseline at $q \leq 8009$: holds (0/5, 0/5 and 2/5 seeds faster; medians

0.71, 0.78, 0.97). Q3 (exploratory): at $q = 12007$ lp:0.5 is faster on 5/5 seeds, median 1.08 (1.06–1.33), outside that q's A/A band (0.968–1.004). This is an exploratory diagnostic, not an established crossover: the runs are wall time on an unisolated shared VM with no reserved or pinned core and no contention admission, and the A/A arm measures repeat spread, not uncontended execution (AGENTS.md requires isolated execution for wall-clock evidence). The crossover stays unestablished until matched isolated measurements support it.

Reading. The precheck is a baseline fix: it removes repeated Wiedemann solves, and on one-round instances it adds only a column scan (11 of 12 such instances were not below the A/A floor). Against the fixed baseline, a single large prime costs more in relation search (2.1–3.0× the candidates at $\alpha = 0.5$) than it saves in linear algebra up to $q \approx 8000$; at $q = 12007$ the unisolated runs suggest it starts to pay, where the baseline's linear algebra is large (exploratory, see Q3). That would match the cost shapes: relation search grows about linearly in q, sparse linear algebra about quadratically. At $q = 12007$ the median distance to rho moves from 4.03 (v1 baseline) to 3.32 (precheck) and 3.08 (lp:0.5 with precheck). Rho is still faster at every q.

Pricing note (AGENTS.md §5a). As in PR #2580: index calculus here does no group operations, so the order of the arms flips across the 0 / 0.1 / 1 GAE band; arms are compared on paired wall time only.

3. Failures and corrections

- Amendment 1: the first v1 run stopped with a panic (the iota-pair check read the log of a point outside the small base). Fixed by computing c from the pinned point; the 18 earlier rows, laid out once, are kept as `results/campaign_v1_superseded.*` and the grid was rerun.
- Amendment 2: the first $q = 12007$ attempt overlapped a test run on the same host; its 3 rows are set aside as `results/campaign_12007_contaminated.jsonl` and $q = 12007$ was rerun alone.

4. Limits

- Toy sizes; 5 instances per q; one host with no isolation, so every wall-time ratio here is exploratory until isolated measurements replicate it.
- Only single large primes; residuals with two outside points are dropped (about 7,800 per instance at $q = 8009$, $\alpha = 0.5$), which a double-large-prime variant could use.
- The validation gate (replay certificate, validator report, `VALIDATION.json`) is not done, so no benchmarks or leaderboard row is added.

5. Records

`PROTOCOL.md`, `PROTOCOL_v2.md`, `results/campaign.jsonl`, `results/v2.jsonl` (+ `.stderr`), `results/summary.md`, `results/summary_v2.md`, `results/summary_v1.csv`, `results/summary_v2.csv`, `results/instances_v1.csv`, `results/instances_v2.csv`, the superseded and contaminated rows. Tables are computed in Rust by `humbert summarize-lp <dir>` (it rejects duplicate records); `plot_lp.py` only draws the figures from those CSVs. Reproduce into a new directory: `run_campaign.sh <new-dir>` and `run_v2.sh <new-dir>` (they refuse an existing directory; `BIN` and `BIN_SHA256_PREFIX` select and check a frozen binary), then `humbert summarize-lp <new-dir>`.